Learning Management System – Final Report (Condensed)

Student name: Ntoetse Selomo · **Student number:** 202321292·

Git repository: https://github.com/ntoetseselomo-crypto/LMS-Phase4

Abstract

This report presents the analysis, design, implementation, testing, deployment and maintenance of a Learning Management System (LMS) for an academic environment. The system manages users, courses, assignments, submissions, marking, moderation, feedback and deadline extensions for students, lecturers, Heads of Department (HoDs), administrators and IT staff. Because computers may be old and internet access unreliable, the application uses C++17 and a local SQLite database rather than a remote server.

The project follows a staged Waterfall model covering planning, requirements, design, implementation, verification and validation, configuration management, deployment and maintenance. The design is layered, separating the user interface, application services, repositories and persistence. The Repository pattern isolates database operations from business logic, and the Strategy pattern supports replaceable late-penalty rules. Passwords are hashed with libsodium's Argon2id, database access uses parameterised SQL, and mark changes are audited.

The initial release was tagged v1.0. After deployment, the school raised change request CR1: lecturers must be able to extend a deadline for a whole class, and students must see the effective deadline on their assignment list. This was classified as perfective maintenance and delivered in release v1.1 through a ClassExtension table, new repository operations and an effective-deadline display. All six CTest groups passed in regression testing, and the GitHub Actions check for pull request #5 passed before the change was merged into main and tagged v1.1.

The report also considers privacy, assessment-record security, accessibility and fairness. The requirements traceability matrix is reserved as Appendix A and must be completed last, once requirements, diagrams, source files and test identifiers have been cross-checked.

1. Introduction

1.1 Background and problem statement

Educational institutions need consistent records of assignments, submissions, marks and deadlines. When these are handled through spreadsheets, paper, email or informal communication, information is duplicated, deadlines become inconsistent, resubmissions are hard to track, and there is little evidence of who changed a mark. HoDs also find it harder to review marks before release. A Learning Management System addresses these problems with role-based functions and a shared local database.

The proposed LMS serves a school or department of about 800 students operating with older computers and unreliable internet access. The case study requires offline operation of core functions, protection of

passwords and assessment records, resubmission before a deadline, written feedback, late-submission rules, and moderation of marks before release.

1.2 Aim and objectives

The aim is to develop a reliable, secure and maintainable LMS that supports the institution's main assignment and assessment activities. The objectives are to:

- provide authenticated access for students, lecturers, HoDs and administrators;
- let authorised staff manage users, courses and assignments;
- let students submit and, where permitted, resubmit work;
- calculate late penalties according to configured rules;
- support marking, written feedback, moderation and approval before release;
- record important mark changes for accountability;
- store data locally in SQLite so core functions work offline;
- support individual extensions and, from v1.1, class-wide extensions;
- provide automated tests and a repeatable build.

1.3 Scope

The system covers authentication, role-based access, course and assignment management, submissions, deadline checking, late penalties, marking, moderation, mark release, feedback, audit logging and extensions. Release v1.1 adds class-wide extensions and shows effective deadlines in the student assignment list. The LMS is a local application, not a cloud platform. Live video teaching, online examinations, mobile apps, cloud synchronisation and integration with external student information systems are out of scope unless separately approved.

1.4 Intended users

User	Main responsibility
Student	View assignments and deadlines, submit and resubmit work, view released marks and feedback.
Lecturer	Create assignments for assigned courses, mark work, give feedback, grant class-wide extensions.
Head of Department	Moderate marks, return work for remarking, approve release.
Administrator	Manage users, courses and enrolments.
IT support	Install, configure, back up and maintain the application and database.

1.5 Report organisation

Sections 2 and 3 cover planning and requirements. Section 4 describes the architecture, classes and database. Sections 5 to 8 cover implementation, testing, configuration management and deployment.

Section 9 documents CR1 and release v1.1, and Section 10 considers professional, legal and ethical issues. The traceability matrix is added last as Appendix A.

2. Project Management Plan

2.1 Development approach

The project uses a staged Waterfall approach. Each phase produces a documented deliverable that informs the next: requirements and use cases precede detailed design, and the design is reviewed before implementation. Waterfall suited the assignment because the case study, deliverables and phase gates are fixed, and it maps directly onto the required report structure. Its limitation is that late changes can force updates to requirements, diagrams, code and tests. CR1 demonstrated this, which is why version control and regression testing were important. **2.2 Work breakdown**

Phase	Main activities	Deliverable
1. Planning	Feasibility, work breakdown, schedule, risk register	Project management plan
2. Requirements	Stakeholder analysis, conflict resolution, SRS, use cases	Requirements package
3. Design	Architecture, UML, patterns, database design	Software design document
4. Implementation	C++17 code, SQLite persistence, security, build setup	Source code and build guide
5. Verification	Test plan, automated and system tests, defects, coverage	Test plan and results
6. Configuration and QA	Branching, commits, CI, review, quality checks	Configuration and QA records
7. Deployment	Installation instructions, user guidance, v1.0 release	Deployment package
8. Maintenance	CR1 analysis, implementation, regression tests, v1.1	Maintenance report and v1.1

2.3 Feasibility study

Technical. The tools suit a small offline application. C++17 supports object-oriented design, SQLite is an embedded database needing no server, CMake and Ninja give a repeatable build, libsodium provides password hashing, and Catch2 with CTest supports automated testing. Development used MSYS2 UCRT64 on Windows with a GNU C++ compiler.

Operational. The system supports existing school roles rather than introducing a new academic process, and role-based menus show each user only relevant functions. Training and clear instructions are still needed for users accustomed to manual processes.

Economic. Open-source tools and an embedded database reduce licensing and server costs, and existing Windows computers can be used if they meet runtime and storage needs. Installation, training, maintenance, backups and replacement of unsupported hardware remain costs to consider before wider deployment.

Legal. The system processes personal and academic records, so it must be operated in line with applicable data-protection requirements in Lesotho and institutional policy. The design limits access by role, avoids plain-text passwords and logs mark changes, but compliance also depends on how the institution collects, retains, shares and deletes information.

Schedule. Phases were sequential, with requirements and design complete before implementation, testing after it, and deployment before the maintenance change. Schedule risk came from database persistence, password hashing, automated tests and Git release management, and was managed by prioritising core requirements and resolving build and test problems early.

2.4 Risk management

Risk	Impact	Mitigation
Requirements misunderstood	Wrong or incomplete features	Review stakeholder needs, document decisions, validate use cases
Database errors or data loss	Lost or corrupted submissions and marks	Transactions, input validation, backups
Unauthorised access	Private data exposed or marks altered	Password hashing, role checks, parameterised SQL
Late requirement changes	Design, code and tests diverge	Change requests, impact analysis, version control, regression tests
Build dependency problems	Application fails to compile elsewhere	Documented dependencies and CMake instructions
Insufficient testing	Defects remain in key workflows	Unit, integration and system tests via CTest and CI
Loss of files or overwrite	Work or releases lost	Regular commits, feature branches, preserved release tags
Users struggle with the system	Poor adoption	Role-based guidance, clear messages, training

For final submission, add likelihood, impact, exposure score, owner and status from the latest risk register (Appendix D).

2.5 Project monitoring

Progress was monitored through phase deliverables, builds, automated tests and Git history. Meaningful commits recorded changes, and feature branches with pull requests allowed maintenance work to be reviewed before merging. Test results showed whether changes introduced regressions, and the v1.1 merge provides a clear point in the history.

3. Requirements Engineering

3.1 Stakeholders, needs and conflicts

The main stakeholders are students, lecturers, HoDs, administrators, IT support and institutional management. Their needs overlap but can conflict.

Stakeholder	Need	Resolution in the design
Students	Submit work, correct mistakes before the deadline, receive feedback, view released marks	Submission and resubmission workflows, deadline checking, feedback and release
Lecturers	Create assignments, mark consistently, apply late rules	Assignment, marking, feedback and extension functions
HoD	Review marks and approve release	Moderation and mark-release controls
Administrator	Maintain users, courses and enrolments	Role-restricted administrative functions
IT officer	Offline operation, protected passwords, acceptable performance on old computers	Local SQLite, password hashing, lightweight design
Management	Reliable records and fair, accountable processes	Audit records, access controls, documented rules

Two conflicts shaped the design. First, rejecting all late work would be simple, but the student representative asked that late work not simply be rejected, so the design uses configured late-penalty rules and extensions. Second, lecturers enter marks while the HoD must review and approve them before release, so marking is separated from moderation and release.

3.2 Functional requirements

The final report should use the exact IDs and wording from the approved SRS.

ID	Requirement
FR-01	The system shall authenticate users using valid login credentials.

ID	Requirement
FR-02	The system shall distinguish Student, Lecturer, HoD and Administrator roles.
FR-03	The system shall restrict functions according to the authenticated user's role.
FR-04	The Administrator shall be able to manage user accounts.
FR-05	The Administrator shall be able to manage courses and enrolments.
FR-06	The Lecturer shall be able to create and manage assignments for authorised courses.
FR-07	The Student shall be able to view assignments relevant to their enrolments.
FR-08	The Student shall be able to submit an assignment before the applicable deadline.
FR-09	The system shall support permitted resubmission before the deadline.
FR-10	The system shall calculate late penalties according to the configured policy.
FR-11	The Lecturer shall be able to mark submissions and provide feedback.
FR-12	The HoD shall be able to moderate marks before they are released.
FR-13	The system shall prevent unauthorised release of marks.
FR-14	The system shall record mark changes in an audit log.
FR-15	Authorised staff shall be able to grant individual deadline extensions.
FR-16	A Lecturer shall be able to grant a class-wide deadline extension.
FR-17	The Student shall be able to see the effective deadline on the assignment list.

3.3 Non-functional requirements

ID	Quality	Requirement / design response
NFR-01	Security	No plain-text passwords; use a recognised hashing function and enforce role checks.
NFR-02	Privacy	Only authorised roles access learner records needed for their duties.
NFR-03	Reliability	Core workflows work without continuous internet; database errors are handled clearly.
NFR-04	Performance	Remain responsive for expected school data volume; measure on target hardware.

NFR-05	Maintainability	Layered components, focused classes, documented interfaces, automated tests.
ID	Quality	Requirement / design response
NFR-06	Usability	Clear menus, validation messages and role-appropriate functions.
NFR-07	Portability	C++17, CMake and documented dependencies for repeatable Windows builds.
NFR-08	Auditability	Important assessment changes traceable to the responsible user and time.

The measurable thresholds approved in the SRS should be retained. A requirement such as response time must be tested on a defined machine and dataset, not declared met because the application seemed responsive in informal use.

3.4 Use case model

The actors are Student, Lecturer, HoD and Administrator. The IT officer is an operational stakeholder rather than a core application role.

- **Student:** log in, view assignment list, submit, resubmit where allowed, view released marks and feedback.
- **Lecturer:** log in, create assignment, view submissions, mark, give feedback, manage deadline extensions.
- **HoD:** review and moderate marks, return work for remarking, approve release.
- **Administrator:** manage users, courses and enrolments.

3.5 Fully dressed use case summaries

Submit assignment (Student). *Preconditions:* authenticated, enrolled, assignment available. *Main flow:* the student selects an assignment, views the applicable deadline, chooses the file and confirms; the system validates the details, checks the effective deadline, applies the late rule if needed, stores the submission and confirms. *Alternative:* where permitted, a replacement submitted before the deadline is recorded under the version policy. *Exception:* invalid file or assignment details, or a storage failure, produce an error and no success message. *Postcondition:* a valid submission with timestamp and status is stored.

Mark submission (Lecturer). *Preconditions:* authenticated, authorised for the course, submission exists. *Main flow:* the lecturer reviews the work, enters a mark and feedback and saves; the system validates the mark and records the change. *Alternative:* the mark may be revised before release, with the change recorded. *Exception:* out-of-range marks and unauthorised requests are rejected. *Postcondition:* mark and feedback are stored for moderation.

Moderate and release marks (HoD). *Precondition:* marks await moderation. *Main flow:* the HoD reviews marks, approves them or returns an item for correction, and authorises release once checks are complete. *Alternative:* a mark needing review is returned with a reason. *Exception:* unauthorised users cannot approve or release. *Postcondition:* approved marks become visible to students only after the release action.

The final submission should include the approved Use Case Diagram and full specifications from the requirements phase; these summaries do not replace them.

4. Software Design Document

4.1 Architecture

The LMS uses a layered architecture, which separates concerns so that one part can change without rewriting the whole system. User-interface code need not contain SQL, repositories can be tested separately from menu handling, and application services are the natural place for validation and role checks.

Layer	Responsibility	Examples
Presentation	Display menus, collect input, show results	main.cpp and menu interaction
Application / service	Coordinate authentication, submissions, marking and rules	AuthService, SubmissionService
Domain	Represent users, assignments, submissions	User, Assignment, Submission
Data access	Database operations and record-to-object mapping	Repositories and interfaces
Persistence	Local relational storage	SQLite database

A client-server architecture was considered and rejected for the current deployment. The case study describes a single-computer environment with unreliable internet, and a separate server would add setup and maintenance burden without helping the core offline workflow. If the institution later needs simultaneous access from several computers, the architecture should be reviewed rather than assuming the local database can safely serve that purpose.

4.2 Design patterns

Repository. Data access is separated from business logic, so a service can fetch an assignment or save a submission without knowing the SQL. Queries stay in a small number of classes, and repository behaviour can be tested through integration tests against SQLite.

Strategy. Late-penalty calculation is a replaceable policy. A strategy computes the penalty for a given lateness period, which keeps the submission service focused on the submission process and lets the

policy be changed or tested separately. Strategy class names should match the source code and class diagram in the final artefacts.

4.3 Object-oriented design

An abstract User concept with role-specific user types lets common identity and account behaviour live in a base class while subclasses add role behaviour. Polymorphism allows code to work with a common User interface, and encapsulation keeps data and operations together. Classes follow the singleresponsibility principle, and dependencies point toward abstractions where practical. Each class should have a clear reason to exist; adding more classes does not itself improve the design.

4.4 Database design

SQLite is the persistence mechanism. Primary keys identify records and foreign keys connect them, and the Enrolment table resolves the many-to-many relationship between students and courses.

Table	Purpose
User	Identity, login-related data and role
Department	Department information
Course	Course details and department link
Enrolment	Student-course association
Assignment	Assessment details, original deadline, maximum mark
Extension	Approved individual deadline extensions
ClassExtension	Class-wide extension introduced for CR1
Submission	Student submission details and status
Mark	Marks and related assessment information
AuditLog	Important mark changes and accountability data

The schema separates related entities to reduce repeated storage and update anomalies, and is intended to approach Third Normal Form. Referential integrity ensures that a submission refers to a valid student and assignment, and that an extension refers to a valid assignment and authorised user. Parameterised queries reduce SQL injection risk. The final database diagram should be checked against the actual CREATE TABLE statements in the source code.

4.5 Class-wide extension data design

CR1 introduced a ClassExtension table holding a primary key, an assignment reference, the new deadline and the granting user. The assignment_id is unique, so each assignment has at most one current classwide extension; granting a new one inserts or updates the record. The Extension table remains for individual student extensions.

The Assignment structure gains an effectiveDeadline value for the deadline applying to a particular student. The repository derives it from the original deadline, any class-wide extension and any individual extension. The student list displays the effective deadline when it is later than the original, and the original deadline remains stored so the assignment's configuration is not lost.

5. Implementation

5.1 Environment

Component	Technology
Language	C++17
Build	CMake with Ninja
Compiler	GNU C++ via MSYS2 UCRT64
Database	SQLite
Password hashing	libsodium Argon2id
Testing	Catch2, run through CTest
Coverage	gcovr / gcov
CI	GitHub Actions
Version control	Git and GitHub

The project is organised into headers (data structures and interfaces), sources (implementations), tests and documentation. CMake describes targets and dependencies; documentation covers design updates, test results, release notes and installation guidance.

5.2 Authentication and password protection

Passwords are never stored in plain text. libsodium's Argon2id function produces a salted, computationally costly hash that is stored and later checked at login. Unlike encryption, hashing is not meant to be reversible, and the salt and cost make large-scale guessing harder. Input is validated before registration or authentication, authentication should return a general failure message rather than reveal whether a username exists, and hashes must not be written to ordinary logs.

5.3 Role-based access control

Student, Lecturer, HoD and Administrator roles have distinct responsibilities. Students cannot create assignments or change marks, lecturers act only on courses they are authorised to teach, HoDs handle moderation and release, and administrators manage users and academic setup. Restrictions are enforced in services and repositories, not only by hiding menu options, because a user-interface restriction can be bypassed if a service method accepts an unauthorised request.

5.4 Assignment and submission processing

A lecturer creates an assignment for an authorised course with a title, maximum mark and deadline. Students see assignments for their enrolled courses. On submission the system validates identifiers and details, records the timestamp and determines whether the work is on time or late. Where policy permits, resubmission is recorded under the submission-history rules.

The late-penalty strategy calculates any penalty. Boundary cases matter: submissions just before, exactly at and just after the deadline may be treated differently under the approved rule. The system uses a consistent date-time representation, validates dates before saving, and returns clear errors for invalid assignment or course identifiers.

5.5 Marking, moderation and audit logging

Lecturers enter marks and written feedback, and marks are validated against the permitted range. A mark is not released merely because it has been entered: HoD moderation provides a further check before results become visible, satisfying the requirement that the HoD review departmental marks first.

Audit records should capture the actor, timestamp, affected record, previous and new values, and a reason where required. They support investigation of mistakes and deter unauthorised changes, and access to them should itself be restricted because they contain sensitive assessment information.

5.6 Database access and error handling

Repository methods centralise database operations and bind user-provided values as parameters rather than concatenating them into SQL. Operations check return codes, handle exceptions and preserve consistency, and transactions group related changes that must succeed or fail together. Input is validated before saving; for example, a malformed extension date is rejected rather than stored. The Phase 5 report records that invalid course IDs and malformed extension dates were tested, corrected and retested. Users receive useful messages without exposure of internal database details.

5.7 CR1 implementation

CR1 adds class-wide extensions without removing individual ones. The repository exposes a method to grant a class-wide extension and a method to retrieve a student's assignments with effective deadlines. The grant method checks that the acting user is a lecturer and that the assignment belongs to a course they are authorised to teach, and the extension is stored separately from individual Extension records.

The student menu retrieves assignments through the repository and displays the effective deadline. If no extension applies, the original deadline stands; if a class-wide or individual extension applies, the later approved deadline is used. Details are in Section 9.

6. Verification and Validation

6.1 Testing approach

Verification checks that the system was built according to its design and requirements; validation checks that it meets users' needs. The project uses unit, integration, system and acceptance testing, related

through the V-model: requirements are checked by acceptance tests, system design by system tests, detailed design by integration tests and components by unit tests.

Level	Purpose	Example
Unit	Test a small function or class in isolation	Late-penalty calculation, input validation
Level	Purpose	Example
Integration	Check connected components together	Repository operations on a temporary SQLite database
System	Exercise complete workflows	Submission, marking and HoD moderation
Acceptance	Confirm user needs and requirements are met	Role-based workflows demonstrated against use cases

6.2 Test design techniques

Equivalence partitioning groups inputs expected to behave alike, such as submission times before, at and after the deadline. Boundary value analysis targets the edges: zero days late, each permitted late day and the first day beyond the permitted period. For extension dates, tests cover a valid future date, a malformed date and a date that breaks the extension rules.

6.3 Automated test environment

Tests use Catch2 and run through CTest, with the build configured by CMake and Ninja on C++17, SQLite and libsodium. The suite has six CTest groups: authentication service, submission service, LMS repository integration, user role conversion, user repository operations and late-penalty rules.

6.4 Phase 5 results

The Phase 5 verification report records that all six automated test groups passed and the GitHub Actions pipeline succeeded. Business-logic line coverage was 96.0%, above the 70% target, and branch coverage of 48.7% was reported separately. Manual testing covered registration, assignment creation, submission, marking, moderation, mark release, audit logging and extension creation.

Two defects were recorded, fixed and retested:

- **DEF-01:** invalid course IDs. The application now shows a clear validation message and prompts for a valid ID.
- **DEF-02:** malformed extension dates. The application now rejects an invalid format before saving, and a valid extension deadline was accepted and checked in SQLite.

6.5 CR1 regression testing

Four regression sections were added for class-wide extensions:

- a lecturer can grant a class-wide extension;

- a new class-wide extension updates the previous one;
- a student's assignment list uses the effective class deadline while preserving the original;
- a user who is not a lecturer is prevented from using the lecturer-only method.

The last section was renamed so its description matches the HoD account used in its fixture. After the project was rebuilt, the full CTest suite showed all six groups passing (100% passed, 0 failed out of 6), and the pull request showed one successful CI check before merging.

6.6 Limitations of test evidence

Passing tests show that the tested behaviours work for the chosen cases; they do not prove the absence of defects. The final submission should attach the complete test output and coverage report (Appendix B) and honestly identify untested requirements. Accessibility and performance in particular should be tested on representative devices and with users rather than assumed from the design.

7. Configuration Management and Quality Assurance

7.1 Version control, branching and commits

Git tracks source code and documentation, and the repository is hosted on GitHub, with main holding the integrated version. The project uses a simple feature-branch workflow: work is done on a separate branch, committed with meaningful messages, pushed, and merged through a pull request. Maintenance work was done on feature/phase8-maintenance and submitted as pull request #5, which allowed review and a CI check before merging. The Phase 8 implementation was committed as 022e13a and a later correction to a test description as c36c682. The merged feature branch could then be deleted.

7.2 Releases and tags

The initial release tags v1.0 and v1.0.0 were preserved. After the merge, local main was updated from origin/main, and an annotated tag v1.1 was created on merge commit 829f2c9d5c1c705f897c1529539256f7f01c796f and pushed to GitHub. This gives a stable reference to the class-wide extension release, supported by release notes describing the feature, database change and verification.

7.3 Continuous integration

GitHub Actions builds the project and runs the automated tests on pushes. A successful CI check was shown on pull request #5 before merging. CI does not replace manual review, but it gives repeatable evidence that the project can be configured, built and tested in a clean environment.

7.4 Quality assurance and configuration items

Quality assurance activities include a consistent project structure, code review, input validation, parameterised SQL, CMake builds, automated tests rerun after each change, and maintained documentation. The final report should include only static-analysis findings and code metrics that were actually measured; if no static analyser was run, that should be stated as a limitation rather than claiming the code passed one.

Configuration items are source and header files, CMake configuration, schema creation statements, tests, installation instructions, design documents and release notes. Changes to these are committed and reviewed. The development database may contain test data and must not be distributed as a production database; a clean installation should create the schema itself.

8. Deployment

8.1 Environment, build and installation

The application was developed and tested on Windows using MSYS2 UCRT64, a GNU C++ compiler, CMake, Ninja, SQLite, libsodium and Catch2, and is built from source. From the project directory:

```
cmake -S . -B build -G Ninja cmake
--build build
```

The executable is then launched from the build directory. Its exact path and runtime dependencies should be confirmed against the final installation guide. A school technician should follow the guide on a clean machine and verify that the application opens, creates or accesses its SQLite database and completes a basic login.

8.2 Data backup and recovery

Because SQLite stores data in a local file, that file must be backed up, either while the application is not writing to it or through an appropriate SQLite backup method. The institution should define backup frequency, storage location, access and restoration testing. A backup that has never been restored is not sufficient evidence of recoverability.

8.3 User guidance

Students are shown how to log in, view assignments and effective deadlines, submit and resubmit, and view released marks and feedback. Lecturers learn to create assignments, review submissions, mark, give feedback and grant class-wide extensions. HoDs learn to moderate and release marks, and administrators to manage users, courses and enrolments. The manual should use screenshots from the final version and describe only features present in the deployed build.

8.4 Releases v1.0 and v1.1

Release v1.0 is the initial deployed system. Release v1.1 is a maintenance release adding class-wide extensions and student visibility of the effective deadline; its tag points to the merged Phase 8 commit on main. The earlier v1.0 tags remain available for comparison or rollback planning.

9. Maintenance: Change Request CR1

9.1 Change request and classification

After the initial release the school submitted CR1: "A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student's assignment list." It extends existing deadline management and does not replace individual extensions.

CR1 is perfective maintenance: it improves an existing feature to meet additional user needs. The original LMS already supported individual extensions, and the request describes no fault in them, so it is not corrective maintenance.

9.2 Impact analysis

Affected area	Change required
Requirements	Add class-wide extensions and student visibility
Use cases	Add lecturer class-wide extensions; retain individual extensions
User interface	Update lecturer extension menu and student assignment list
Repository	Add grantClassExtension() and assignmentsForStudent()
Assignment data	Add effectiveDeadline
Database	Add ClassExtension; retain Extension
Security	Check the user is a lecturer authorised for the assignment's course
Tests	Cover creation, update, effective deadline display and access control

9.3 Implementation

The schema creates a ClassExtension table storing the assignment identifier, new deadline and granting user. The unique assignment reference keeps one current class-wide extension per assignment. grantClassExtension validates the user's role and course authority before inserting or updating, which prevents a lecturer extending an assignment outside their teaching responsibility.

assignmentsForStudent retrieves a student's assignments and calculates each effective deadline from the original deadline, any class-wide extension and any individual extension. The student menu displays it when an extension changes the original. Individual extensions remain available through the authorised HoD and Administrator flow.

9.4 Regression testing and release

Because the new feature interacts with individual extensions and the student list, four test sections were added to the repository integration tests. After the final test-description correction the build completed and all six CTest groups passed, and pull request #5 showed a successful CI check before merging into main.

Commits were made on feature/phase8-maintenance and reviewed through pull request #5. The merge commit is 829f2c9d5c1c705f897c1529539256f7f01c796f. The annotated tag v1.1, with the message "LMS version 1.1 - Class-wide deadline extensions", was pushed to GitHub, keeping the release identifiable and the earlier v1.0 tags intact.

9.5 Limitations

ClassExtension stores one current class-wide deadline per assignment. If the institution needs a history of class-wide decisions, the schema may need an extension-history table or audit records for extension changes. Policy should also state how an individual extension interacts with a class-wide one. The current logic selects the later effective deadline, and the institution should confirm that this is the intended rule.

10. Professional, Legal and Ethical Issues

10.1 Data protection and privacy

The LMS stores personal information about learners and staff, including names, account identifiers, login-related data, enrolments, submissions, marks and feedback. It must be collected and used for legitimate educational purposes and disclosed only to those who need it. For use in Lesotho, the institution should consider the Data Protection Act 2011 (Act No. 5 of 2012), applicable regulations, institutional policy and later amendments.

The design supports privacy through role-based access, Argon2id password hashing and controlled repository operations. The application should collect only what the LMS functions need, restrict access to assessment records and keep sensitive information out of logs. The institution remains responsible for retention periods, privacy notices, data-subject requests and protection of backups.

10.2 Security of assessment records

Marks, submissions and feedback affect academic progress, so unauthorised changes could cause unfair outcomes. The system restricts actions by role, validates marks, requires moderation before release, uses parameterised SQL and logs mark changes so unexpected updates can be investigated.

Security also depends on operational practice: disabling or removing accounts no longer needed, using strong unshared passwords, setting file-system permissions on databases and backups, protecting computers from physical access, and keeping backup copies secure.

10.3 Accessibility

The LMS should be usable by people with different abilities. Visually impaired users need readable text, sufficient contrast, clear labels and screen-reader compatibility; users with motor impairments need keyboard navigation and controls that do not demand precise mouse movement; hearing-impaired users should not depend on audio-only instructions; and clear language and consistent navigation help users with cognitive or learning difficulties.

Accessibility is a design and testing requirement, not an assumption. The interface should have meaningful labels, logical focus order, readable messages and adequate contrast, and be tested with keyboard-only navigation and, where possible, assistive technologies and users with disabilities. Features not yet implemented should be recorded as future work, not claimed as complete.

10.4 Fairness of late penalties

Late penalties affect results, so they must be clear, consistent and applied under an approved policy. The system uses a configured penalty strategy rather than letting staff apply arbitrary penalties, and students can see the applicable deadline, including approved extensions. Class-wide and individual extensions support fairness where extra time has been authorised.

The institution should publish its late-submission policy before assignments are due, covering the permitted late period, penalty calculation, rounding, technical problems and how to request an extension, with a documented process for exceptional circumstances. The LMS can apply rules consistently, but whether a policy is fair remains an institutional responsibility.

10.5 Professional responsibilities and ethical use

Developers should produce understandable, maintainable and reasonably secure code, should not claim a feature is tested without evidence, should review, document and test changes, should respect learner confidentiality and should report serious security or data-integrity problems. User documentation must describe the released application accurately.

The system should support learning rather than unfairly monitor or disadvantage learners. Staff should access only the records they need, marking and moderation decisions should be explainable, and learners should receive clear information about deadlines, feedback and results. Audit records support accountability but should not be used to monitor staff or students beyond legitimate institutional purposes.

11. Conclusion

The project delivered a local application supporting core academic administration: authentication, rolebased access, course and assignment management, submissions, late penalties, marking, moderation, feedback, audit logging and deadline extensions. C++17, SQLite, CMake, libsodium and Catch2 were chosen to give a maintainable offline application with automated verification.

The layered architecture separates interaction, services, repositories and persistence. The Repository pattern centralises database access and the Strategy pattern separates late-penalty rules from the submission workflow. Password hashing, role-based access, parameterised SQL and audit logging address the main security concerns. The verification report records six passing automated test groups, successful CI and 96.0% business-logic line coverage.

CR1 was implemented as perfective maintenance in release v1.1: lecturers can grant class-wide extensions and students see the effective deadline, while the original deadline and individual extensions are retained. The change was tested, merged through pull request #5 and tagged v1.1. As the system moves toward real institutional use, accessibility, performance, data-protection practice and policy clarity should continue to be reviewed.

The requirements traceability matrix remains to be added. It should be prepared last, linking final requirement IDs to approved use cases, diagrams, classes, database tables, source files and test cases, with every link checked against the repository and final artefacts.

12. References

1. Government of Lesotho. (2011). Data Protection Act, 2011 (Act No. 5 of 2012). Verify the official Gazette citation and current amendments before submission.
2. International Organization for Standardization. (2011). ISO/IEC/IEEE 29148:2011 Systems and software engineering: Life cycle processes: Requirements engineering.
3. Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1.
4. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
5. SQLite. (n.d.). SQLite documentation. https://www.sqlite.org/docs.html 6. libsodium. (n.d.). Password hashing documentation. https://doc.libsodium.org/password_hashing

7. C++ Standards Committee. (2017). ISO/IEC 14882:2017 Programming languages: C++.
8. Catch2. (n.d.). Catch2 documentation. https://catch2.github.io/
9. CMake. (n.d.). CMake documentation. https://cmake.org/documentation/
10. GitHub. (n.d.). GitHub Actions documentation. https://docs.github.com/actions

Use one referencing style throughout, confirm legislation details against official sources, and add any textbooks, standards and institutional policies actually used.

13. Appendices (to be completed)

- **Appendix A: Requirements traceability matrix.** Reserved for completion last. It should link every functional requirement to its use case, design elements, database tables, source files and test cases. Do not submit this placeholder as the completed matrix.
- **Appendix B: Full automated test results.** Attach the final CTest output showing all six groups passed, the GitHub Actions evidence and the coverage report (96.0% business-logic line coverage reported in Phase 5).
- **Appendix C: User manuals.** Student, Lecturer, HoD and Administrator instructions, including lecturer class-wide extension management and the student's effective-deadline view.
- **Appendix D: Updated risk register.** Latest register with likelihood, impact, exposure, mitigation, owner and status, including new risks from implementation, testing, deployment and CR1.
- **Appendix E: Release evidence.** Repository link, pull request #5, successful CI result, v1.1 tag and release notes. The v1.1 tag points to merge commit 829f2c9d5c1c705f897c1529539256f7f01c796f.